

Express.js Form Validation Using Express-Validator

A Complete Tutorial (Summary)

1. Introduction to Form Validation

Purpose of Validation

- Ensure users submit correct and safe data.
- Prevent submission of:
 - Empty fields
 - Incorrect data types
 - Invalid email formats
 - Script injections (XSS)
 - Manipulated or tampered values (via browser dev tools)

Real-world threats without validation

- Bypassing input types (e.g., changing `type="number"` to text)
- Injecting JavaScript into fields
- Entering invalid or unexpected values in dropdowns
- Submitting excessive characters or wrong formats

Validation must always happen on **backend**, even if frontend validation exists.

2. Installing and Importing Express-Validator

Installation Command

```
npm install express-validator
```

Importing in Your Express Application

```
const { body, validationResult } = require('express-validator');
```

These two methods are essential:

- **body()** → defines validation rules on request body fields.
 - **validationResult()** → extracts errors after validations run.
-

3. Creating Validation Rules

Use an array to define all rules for a particular form.

Example Validation Rule Array

```
const validationRegistration = [  
  body('username')  
    .notEmpty().withMessage('Username is required')  
    .isLength({ min: 3 }).withMessage('Username must be at least 3 characters  
long'),  
  
  body('email')  
    .isEmail().withMessage('Please provide a valid Email ID'),  
  
  body('password')  
    .isLength({ min: 5, max: 10 })  
    .withMessage('Password must be between 5 and 10 characters long'),  
  
  body('age')  
    .isNumeric().withMessage('Age must be numeric')  
];
```

4. Applying Validations Using Middleware

Validation arrays must be added as Express middleware in the route:

```
app.post('/save-form', validationRegistration, (req, res) => {  
  const errors = validationResult(req);  
  
  if (!errors.isEmpty()) {  
    return res.send(errors.array());  
  }  
  
  // Proceed if no errors  
  res.send(req.body);  
});
```

5. Understanding Express-Validator Methods

Common Validator Methods

Method	Purpose
<code>notEmpty()</code>	Field must not be empty
<code>isEmail()</code>	Valid email format
<code>isLength({min,max})</code>	Character length restriction
<code>isNumeric()</code>	Only digits allowed
<code>isAlpha()</code>	Alphabets only
<code>isAlphanumeric()</code>	Alphabets + numbers
<code>isURL()</code>	Valid URL check
<code>isDate()</code>	Valid date check
<code>isIn([arr])</code>	Value must match one item from given array
<code>isStrongPassword()</code>	Password complexity rules
<code>isUppercase()</code>	Uppercase only
<code>isLowercase()</code>	Lowercase only
<code>matches(regex)</code>	Custom regex validation

6. Sanitization Methods

Sanitizers modify values before saving to database.

Common Sanitizers

Method	Purpose
<code>trim()</code>	Removes leading/trailing spaces
<code>ltrim()</code>	Removes left spaces
<code>rtrim()</code>	Removes right spaces
<code>escape()</code>	Prevent XSS by encoding HTML entities
<code>normalizeEmail()</code>	Converts domain to lowercase, removes invalid chars
<code>toInt()</code>	Convert to integer
<code>toFloat()</code>	Convert to float
<code>toBoolean()</code>	Convert value to <code>true/false</code>

7. Practical Validation Examples

7.1 Validating Name Field

```
body('firstname')
  .notEmpty().withMessage('First Name is required')
  .trim()
  .isLength({ min: 5 }).withMessage('First Name must have 5 characters')
  .escape()
  .isAlpha().withMessage('First Name has alpha characters only')
```

7.2 Strong Password Validation

```
body('password')
  .isStrongPassword()
  .withMessage('Password must be strong')
```

7.3 Validating Age Minimum

```
body('age')
  .isInt({ min: 18 })
  .withMessage('Age must be at least 18 years old')
```

7.4 Restricting to Selected City Values

```
body('user_city')
  .isIn(['Delhi', 'Mumbai', 'Goa', 'Agra'])
  .withMessage('Please select a valid city')
```

8. Display Errors in an EJS Template

In Route: Send Errors to View

```
res.render('myform', { errors: errors.array() });
```

In `myform.twig`

```
{% if errors|length > 0 %}
  <div class="alert alert-danger">
    <ul style="list-style-position: inside;">
      {% for err in errors %}
        <li>{{ err.msg }}</li>
      {% endfor %}
    </ul>
  </div>
{% endif %}
```

9. Creating Custom Validations

Custom Rule (e.g., "admin" username not allowed)

```
body('username')
  .custom(value => {
    if (value.toLowerCase() === 'admin') {
      throw new Error('Username "admin" is not allowed');
    }
    return true;
  });
```

10. Creating Custom Sanitizers

Example: Convert Username to Lowercase

```
body('username')
  .customSanitizer(value => value.toLowerCase());
```

11. Summary of Complete Validation Workflow

1. Install Express-Validator.
 2. Import `body` and `validationResult`.
 3. Build array of validation and sanitization rules.
 4. Attach rules as middleware to your POST route.
 5. On failure: extract `errors.array()` and show in UI.
 6. On success: process or save sanitized data.
 7. Optionally add custom validators or sanitizers for special rules.
-